

Lab 2 - Multithreaded programming with OpenMP

Oguz Kaya

oguz.kaya@universite-paris-saclay.fr

Environment. As in lab 1, we use Compiler Explorer (<https://godbolt.org>) to write, compile and run the programs: language C++, a recent x86-64 gcc, the compilation options given below in the “Compiler options” field, and “Execute the code” ticked in the “Output...” menu of the compiler pane. Save your code regularly in a local file. Compiler Explorer may run your program on a single core, in which case the threads take turns and no speedup can be observed; for the timing questions, compile and run the same file on your own machine:

```
g++ -O2 -fopenmp program.cpp -o program then ./program
```

On macOS, install gcc with Homebrew (`brew install gcc`) and use `g++-16` in place of `g++` (on macOS, `g++` secretly runs the `clang++` compiler, which lacks OpenMP). On Windows, use WSL to install Ubuntu, then compile with `g++` as above. The compilation option `-fopenmp` is what turns the `#pragma omp` directives into threads: without it, they are silently ignored and the program runs sequentially.

Submission. For the questions that ask for a comment, an execution or a result (timings, speedups, explanations), write your answers as comments at the top of the corresponding source file, in the block provided for this purpose; the source files are the only thing you submit.

Multithreaded programming with OpenMP. In this lab, we write our first multithreaded programs, using the minimal toolbox of the lecture: a parallel region creates P threads that all execute the same block; a thread knows its id and the number of threads; the threads are asynchronous, and are synchronized at the end of the region and at barriers; a `sections` block distributes independent pieces of work among the threads.

<code>#pragma omp parallel num_threads(P) { ...}</code>	the block is executed by P threads; implicit barrier at the end
<code>int omp_get_thread_num()</code>	id of the calling thread, from 0 to $P - 1$
<code>int omp_get_num_threads()</code>	number of threads of the current region
<code>#pragma omp barrier</code>	no thread goes on before all the threads of the region are here
<code>#pragma omp sections { ...}</code>	contains <code>section</code> blocks; implicit barrier at the end
<code>#pragma omp section { ...}</code>	executed exactly once, by one of the threads

These are the only OpenMP constructs and functions you are allowed to use in this lab; they are declared in the header `<omp.h>`. For timing, the skeleton codes provide a function `now()` that returns the current time in seconds (based on `std::chrono`), as in lab 1.

Remember the two rules for variables: a variable declared *before* the parallel region is shared by all threads (one copy); a variable declared *inside* the region, or inside a section, belongs to the thread that executes it (one copy per thread). And the one rule for correctness: **two threads must never write the same shared variable at the same time**. When several threads contribute to one result (a sum, a maximum, ...), each thread accumulates its contribution in a variable of its own, stores it into a cell of its own (for instance `partial[thid]`), and the partial results are merged sequentially after the region.

Part 1

Hello world with threads

The goal of this exercise is to create threads and to see what each of them does. A skeleton code is provided in the file `hello.cpp`.

Ex. 1

- Write a parallel region with 4 threads in which each thread prints the line “I am thread <id> of <n>”, where <n> is the number of threads, using `omp_get_thread_num()` and `omp_get_num_threads()`.
- In the same parallel region, make the master thread (the thread whose id is 0) print “Hello from the master thread, id = 0” (a thread can test its own id). Make sure that this line *always* comes after all the lines of question (a).

- c) Still in the same region, make *one* thread, chosen by OpenMP, print “Hello from a random thread, id = <id>” (hint: use `sections`). Make sure that this line *always* comes after the master’s line.
- d) Run the program several times, then with 2 and with 8 threads. What changes in the output, and what does not? What could happen without the constructs you added to enforce the order in questions (b) and (c)?

Part 2

Sum of an array with sections

The goal of this exercise is to distribute a computation on an array among threads with `sections`, and to measure the speedup. A skeleton code is provided in the file `sum.cpp`; it allocates an array `A` of $N = 10^7$ double precision numbers, and contains the timing and printing code.

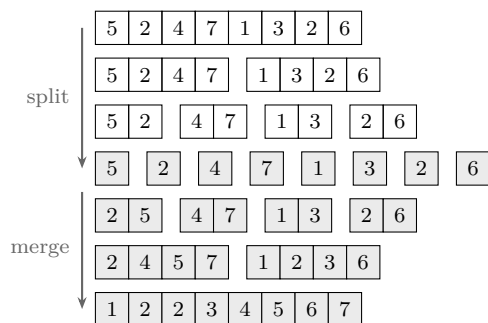
Ex. 2

- a) Initialize `A` such that `A[i] = i` for all $0 \leq i < N$, with a sequential loop.
- b) Add a second sequential loop that computes the sum of all the elements of `A`. With this initialization the sum is $N(N-1)/2$, which the skeleton prints for verification.
- c) Initialize `A` again, this time in a parallel region with 4 threads, using a `sections` block with 4 sections, each initializing $N/4$ elements.
- d) In the same parallel region, after the initialization sections, add a second `sections` block with 4 sections that computes the sum: each section adds its $N/4$ elements into a local accumulator (a variable declared inside the section), then stores it into its own cell of the array `partial`. After the parallel region, merge the 4 partial sums into the final sum, sequentially. Note that the implicit barrier at the end of the first `sections` block guarantees that `A` is fully initialized before the second block starts.
- e) On your own machine, run the program with `num_threads(1)`, (2), (4) and (8), and compute the speedup and the efficiency for each number of threads. Why does the time no longer improve beyond 4 threads?

Part 3

Parallel mergesort with sections

Mergesort is a classical sorting algorithm based on the divide-and-conquer principle: to sort an array of n elements, sort each of its two halves (recursively, with the same method), then *merge* the two sorted halves into a single sorted array. Merging two sorted arrays is easy: repeatedly take the smaller of the two front elements. An array of 0 or 1 element is already sorted, which stops the recursion. The figure shows the two phases on 8 elements: the array is split down to single elements, then the pieces are merged back, two by two, into larger and larger sorted arrays.



- 1: **procedure** MERGESORT(a, n) $\blacktriangleright$ sorts $a[0 \dots n]$
- 2: **if** $n < 2$ **then return** $\blacktriangleright$ already sorted
- 3: MERGESORT($a, n/2$) $\blacktriangleright$ left half $a[0 \dots n/2]$
- 4: MERGESORT($a + n/2, n - n/2$) $\blacktriangleright$ right half
- 5: MERGE(a, n) $\blacktriangleright$ merge the two sorted halves

Merging n elements costs $O(n)$ operations, and each of the $\lceil \log_2 n \rceil$ levels of recursion merges n elements in total: mergesort sorts n elements in $O(n \log n)$ time.

The two halves (or the four quarters) of the array can be sorted independently of each other, hence in parallel; only the merges remain to be done afterwards. The goal of this exercise is to exploit this to sort an array of N integers in parallel, using `sections`. A skeleton code is provided in the file `mergesort.cpp`. It allocates an array `a` of $N = 2^{22}$ integers, fills it with pseudo-random values, and provides three functions:

- `mergesort(a, temp, n)` sorts the n elements of the array `a` sequentially, with the algorithm above, using `temp`, a scratch buffer of n elements;
- `merge(a, temp, n)` takes an array `a` of n elements whose two halves `a[0..n/2)` and `a[n/2..n)` are *already sorted*, and rearranges `a` into a single sorted array of n elements, using `temp` as scratch buffer;
- `verify(a, n, name)` checks that the array is sorted.

The skeleton first sorts the array with the sequential `mergesort` and prints its time; you then sort the same array again, in parallel, in three phases. Recall that `a + b` is the address of `a[b]`, so that the part of `a` starting at index `b` can be passed to these functions as an array of its own (and likewise for `temp`).

Ex. 3

- a) Create a parallel region with 4 threads and a `sections` block with 4 sections; each section sorts one quarter of the array ($N/4$ elements) with the given `mergesort` function.
- b) In the same parallel region, after the sorting sections, add a second `sections` block with 2 sections: each section merges two sorted quarters into a sorted half of $N/2$ elements, with `merge`.
- c) After the parallel region, merge the two sorted halves into the final sorted array of N elements, again with `merge`. Check the verdict of `verify`.
- d) On your own machine, compare the sequential and the parallel times with 4 threads: what is the speedup, what is the efficiency? Why is the speedup below 4, even on 4 cores? If your machine has at least 8 cores, modify your code (in the same file `mergesort.cpp`) so that it works with 8 threads instead, and measure again; this should be a quick modification once the 4-thread version works.